

Backlog produktu Calisia po zakończeniu Sprintu 6

Założenia

Calisia to aplikacja bankowości internetowej dla klientów indywidualnych. Backlog został przygotowany na podstawie istniejącego kodu: backendu .NET z warstwami Api, Application, Domain, Infrastructure, Contracts i ReadModel, wzorca CQRS, osobnych baz CalisiaWriteDb i CalisiaReadDb, projekcji dashboardu, frontendu React/Vite oraz kolekcji Bruno.

Estymacja używa ciągu Fibonacciego w zakresie 0-13 SP: 0, 1, 2, 3, 5, 8, 13.

Uwaga techniczna: w części kodu widoczne są nazwy przestrzeni **Fortuna**, natomiast dokumentacja i nazwa produktu używają marki Calisia.

Część 1 - lista epików

ID	Epik	Opis wartości biznesowej
E01	Fundamenty techniczne aplikacji	Przygotowanie rozwiązania backendowego, frontendu, baz danych, konfiguracji, dokumentacji i narzędzi developerskich.
E02	Rejestracja i tożsamość klienta	Klient może utworzyć profil, a system bezpiecznie zapisuje dane i hasło.
E03	Logowanie, sesja i bezpieczeństwo API	Klient loguje się do bankowości, a chronione operacje wymagają poprawnego tokenu JWT.
E04	Produkty i rachunki bankowe	Klient może otworzyć rachunek lub dodać produkt finansowy, który staje się widoczny w systemie.
E05	Operacje finansowe i przelewy	Klient może zasilać konto, wypłacać środki oraz wykonywać przelewy własne i zewnętrzne.
E06	Dashboard i model odczytowy	Klient widzi saldo, produkty i najnowsze zdarzenia na szybkim dashboardzie opartym o model odczytowy.
E07	Stabilizacja, testy i odbiór produktu	Produkt jest sprawdzony testami automatycznymi, manualnie w UI i przez Bruno, a dokumentacja pozwala go uruchomić i zaprezentować.

Część 2 - podział na 7 sprintów

Rejestr historyjek produktu

Statusy historyjek mogą przyjmować wartości: **to do**, **in progress**, **done**.

ID	Sprint	Epik	Historyjka	SP	Status
S1-01	Sprint 1	E01	Struktura rozwiązania backendowego	8	done
S1-02	Sprint 1	E01	CQRS, obsługa wyjątków i kontrakty API	8	done
S1-03	Sprint 1	E01	Frontend React/Vite i podstawowa nawigacja	5	done
S1-04	Sprint 1	E01	Bazy danych write/read	5	done
S2-01	Sprint 2	E02	Rejestracja profilu klienta	8	done
S2-02	Sprint 2	E02	Walidacja danych rejestracyjnych	5	done
S2-03	Sprint 2	E02	Bezpieczne hashowanie haseł	5	done
S2-04	Sprint 2	E02	Kontrakt rejestracji i dokumentacja requestu	3	done

ID	Sprint	Epik	Historyjka	SP	Status
S3-01	Sprint 3	E03	Logowanie klienta	8	done
S3-02	Sprint 3	E03	Token JWT i dane sesji	8	done
S3-03	Sprint 3	E03	Autoryzacja chronionych endpointów	5	done
S3-04	Sprint 3	E03	Spójna obsługa błędów logowania	3	done
S4-01	Sprint 4	E04	Otwarcie rachunku bankowego	8	done
S4-02	Sprint 4	E04	Dodawanie produktu finansowego	8	done
S4-03	Sprint 4	E05	Wypłata środków z rachunku	8	done
S4-04	Sprint 4	E05	Wpłata przychodząca i blokada niedozwolonego salda	8	done
S5-01	Sprint 5	E06	Endpoint dashboardu klienta	8	done
S5-02	Sprint 5	E06	Kafelki produktów i saldo łączne w UI	5	done
S5-03	Sprint 5	E06	Projekcje zdarzeń domenowych do read modelu	13	done
S5-04	Sprint 5	E06	Oś czasu zdarzeń w dashboardzie	5	done
S6-01	Sprint 6	E05	Przelew własny między rachunkami klienta	13	done
S6-02	Sprint 6	E05	Przelew zewnętrzny	13	done
S6-03	Sprint 6	E05, E06	Panel przelewów i szybka aktualizacja dashboardu	8	done
S6-04	Sprint 6	E05	Walidacje i scenariusze negatywne przelewów	8	done
S7-01	Sprint 7	E07	Pełny przepływ klienta w UI	8	to do
S7-02	Sprint 7	E07	Responsywność i spójność wizualna	5	to do
S7-03	Sprint 7	E07	Rozszerzenie testów automatycznych backendu	13	to do
S7-04	Sprint 7	E07	Kolekcja Bruno i testy manualne API	5	to do
S7-05	Sprint 7	E07	Build, dokumentacja i scenariusz prezentacji	5	to do

Sprint 1 - fundamenty architektury i środowiska

Cel sprintu: przygotować techniczny szkielet aplikacji, aby zespół mógł rozwijać backend, frontend, bazy danych i testy w jednym spójnym projekcie.

S1-01 - Struktura rozwiązania backendowego

Jako zespół chcemy mieć warstwową solucję backendową, aby oddzielić API, logikę aplikacyjną, domenę, infrastrukturę, kontrakty i model odczytowy.

Epik: E01 Story points: 8

Zadania:

- Backend: utworzyć projekty `Calisia.Api`, `Calisia.Application`, `Calisia.Domain`, `Calisia.Infrastructure`, `Calisia.Contracts`, `Calisia.ReadModel`.
- Backend: dodać referencje między projektami zgodnie z kierunkiem zależności warstw.
- Backend: skonfigurować dependency injection dla `Application`, `Infrastructure` i `ReadModel`.
- Backend: dodać endpoint `/api/health` do podstawowego sprawdzania działania API.
- Testy integracyjne: przygotować bazowy projekt `Calisia.IntegrationTests` oraz test endpointu `health`.

- Testy manualne backend/Bruno: dodać lub opisać request sprawdzający health API.

S1-02 - CQRS, obsługa wyjątków i kontrakty API

Jako developer chcę mieć spójny sposób obsługi komend, zapytań i błędów, aby nowe funkcje powstawały według jednego wzorca.

Epik: E01 Story points: 8

Zadania:

- Backend: przygotować interfejsy `ICommand`, `ICommandHandler`, `IQuery`, `IQueryHandler`.
- Backend: dodać middleware obsługi wyjątków i mapowanie błędów domenowych, walidacyjnych, not found oraz unauthorized.
- Backend: zdefiniować pierwsze kontrakty request/response w projekcie `Calisia.Contracts`.
- Backend: skonfigurować OpenAPI/Swagger dla endpointów.
- Testy jednostkowe: pokryć podstawowe zachowanie dispatcherów lub handlerów bez zależności infrastrukturalnych.
- Testy manualne backend/Bruno: sprawdzić odpowiedzi błędów dla niepoprawnych requestów.

S1-03 - Frontend React/Vite i podstawowa nawigacja

Jako klient chcę wejść na stronę startową i przechodzić do logowania lub tworzenia konta, aby rozpocząć pracę z aplikacją.

Epik: E01 Story points: 5

Zadania:

- Frontend: skonfigurować projekt React/Vite, TypeScript, routing i podstawowe layouty publiczne oraz dashboardowe.
- Frontend: przygotować strony `LandingPage`, `LoginPage`, `CreateAccountPage`, `DashboardPage`.
- Frontend: dodać globalne style, reset CSS i współdzielone komponenty `Button`, `Input`, `InfoPopup`.
- Frontend: skonfigurować klienta API z bazowym adresem backendu.
- Testy manualne frontend: sprawdzić przejścia między ścieżkami `/`, `/login`, `/create-account`, `/dashboard`.
- Testy techniczne: uruchomić `npm run lint` i `npm run build`.

S1-04 - Bazy danych write/read

Jako zespół chcemy mieć osobne bazy operacyjne i odczytowe, aby oddzielić zapis transakcji od prezentacji dashboardu.

Epik: E01 Story points: 5

Zadania:

- Baza danych: przygotować skrypt `CalisiaWriteDb.sql` dla tabel klientów, produktów, rachunków, transakcji, przelewów i outboxa.
- Baza danych: przygotować skrypt `CalisiaReadDb.sql` dla kafelków produktów, osi czasu i przetworzonych wiadomości.
- Backend: skonfigurować `WriteDbContext` oraz `ReadDbContext`.
- Backend: dodać konfigurację EF Core dla encji domenowych i modeli odczytowych.
- Testy integracyjne: zweryfikować połączenie z kontekstem testowym.
- Dokumentacja: opisać uruchomienie skryptów baz danych w README.

Suma sprintu: 26 SP

Sprint 2 - rejestracja klienta

Cel sprintu: umożliwić klientowi założenie profilu oraz bezpieczne zapisanie danych logowania.

S2-01 - Rejestracja profilu klienta

Jako klient chcę zarejestrować profil, podając imię, nazwisko, e-mail, hasło i typ klienta, aby rozpocząć korzystanie z bankowości.

Epik: E02 Story points: 8

Zadania:

- Backend: zaimplementować encje i value objecty klienta, w tym `Customer`, `CustomerId`, `FullName`, `Email`, `CustomerType`.
- Backend: dodać `RegisterCustomerCommand`, handler oraz repozytorium klienta.
- Backend: udostępnić endpoint `POST /api/customers`.
- Baza danych: dodać mapowanie i tabelę klienta z unikalnym adresem e-mail.
- Frontend: przygotować formularz tworzenia konta i wysłanie danych do API.
- Testy jednostkowe: pokryć poprawną rejestrację i podstawowe reguły domenowe klienta.
- Testy integracyjne: sprawdzić rejestrację przez API.
- Testy manualne frontend/Bruno: wykonać rejestrację z UI i request `register-customer.bru`.

S2-02 - Walidacja danych rejestracyjnych

Jako system chcę odrzucać niepoprawne dane klienta, aby w bazie znajdowały się tylko poprawne profile.

Epik: E02 Story points: 5

Zadania:

- Backend: dodać walidację pustych pól imienia, nazwiska, e-maila i hasła.
- Backend: dodać walidację formatu e-maila i obsługę zduplikowanego adresu.
- Backend: zwracać czytelne błędy przez middleware wyjątków.
- Frontend: wyświetlać komunikaty walidacyjne przy formularzu rejestracji.
- Testy jednostkowe: dodać przypadki niepoprawnego e-maila, pustego hasła i duplikatu.
- Testy integracyjne: sprawdzić statusy odpowiedzi API dla błędnych danych.
- Testy manualne frontend/Bruno: wykonać negatywne scenariusze rejestracji.

S2-03 - Bezpieczne hashowanie haseł

Jako system chcę hashować hasła PBKDF2, aby nie przechowywać haseł w postaci jawnej.

Epik: E02 Story points: 5

Zadania:

- Backend: przygotować interfejs `IPasswordHasher`.
- Backend: zaimplementować `Pbkdf2PasswordHasher`.
- Backend: zintegrować hasher z handlerem rejestracji.
- Baza danych: zapisać hash hasła w danych klienta.
- Testy jednostkowe: sprawdzić poprawną weryfikację hasła i odrzucenie błędnego hasła.
- Testy manualne backend/Bruno: potwierdzić, że rejestracja działa po zmianie mechanizmu hashowania.

S2-04 - Kontrakt rejestracji i dokumentacja requestu

Jako integrator chcę mieć stabilny kontrakt rejestracji, aby frontend i Bruno używały tego samego formatu danych.

Epik: E02 Story points: 3

Zadania:

- Backend: doprecyzować `RegisterCustomerRequest` i `RegisterCustomerResponse`.
- Backend: zapewnić zwrot identyfikatora klienta po poprawnej rejestracji.
- Frontend: dostosować typy TypeScript do kontraktu API.
- Bruno: uzupełnić request rejestracji przykładowymi danymi.
- Testy integracyjne: sprawdzić strukturę odpowiedzi API.
- Dokumentacja: opisać minimalny payload rejestracji.

Suma sprintu: 21 SP

Sprint 3 - logowanie, sesja i ochrona API

Cel sprintu: zapewnić mechanizm uwierzytelniania oraz zabezpieczyć operacje klienta przed dostępem anonimowym.

S3-01 - Logowanie klienta

Jako klient chcę zalogować się e-mailem i hasłem, aby uzyskać dostęp do bankowości internetowej.

Epik: E03 Story points: 8

Zadania:

- Backend: dodać `LoginCustomerCommand` i handler logowania.
- Backend: porównać hasło z haszem zapisanym przy rejestracji.
- Backend: udostępnić endpoint `POST /api/customers/login`.
- Frontend: przygotować formularz logowania z obsługą stanu ładowania i błędów.
- Frontend: po poprawnym logowaniu przekierować klienta do dashboardu.
- Testy jednostkowe: pokryć poprawne logowanie i błędne hasło.
- Testy integracyjne: sprawdzić logowanie przez API.
- Testy manualne frontend/Bruno: wykonać request `login-customer.bru` i logowanie w UI.

S3-02 - Token JWT i dane sesji

Jako system chcę generować token JWT z identyfikatorem klienta, aby frontend mógł wykonywać autoryzowane zapytania.

Epik: E03 Story points: 8

Zadania:

- Backend: przygotować `ITokenProvider` oraz `JwtTokenProvider`.
- Backend: dodać konfigurację issuer, audience, signing key i czasu życia tokenu.
- Backend: umieścić w tokenie claim identyfikatora klienta.
- Frontend: zapisać token i dane klienta w stanie aplikacji lub sesji.
- Frontend: dołączać nagłówek `Authorization: Bearer` w autoryzowanych requestach.
- Testy jednostkowe: sprawdzić generowanie tokenu i claimy.
- Testy integracyjne: sprawdzić dostęp z poprawnym tokenem.
- Testy manualne backend/Bruno: skopiować token z logowania do chronionych requestów.

S3-03 - Autoryzacja chronionych endpointów

Jako klient chcę, aby moje rachunki, produkty, dashboard i przelewy były dostępne tylko po zalogowaniu.

Epik: E03 Story points: 5

Zadania:

- Backend: oznaczyć endpointy rachunków, produktów, przelewów i dashboardu atrybutem `Authorize`.
- Backend: dodać helper pobierający wymagany `CustomerId` z claims.
- Backend: zwrócić `Forbidden` przy próbie pobrania dashboardu innego klienta.
- Frontend: obsłużyć brak sesji i przekierowanie do logowania.
- Testy integracyjne: dodać scenariusze 401 i 403.
- Testy manualne backend/Bruno: uruchomić requesty bez tokenu i z tokenem innego klienta.

S3-04 - Spójna obsługa błędów logowania

Jako klient chcę widzieć zrozumiały komunikat przy nieudanym logowaniu, aby poprawić dane i spróbować ponownie.

Epik: E03 Story points: 3

Zadania:

- Backend: ujednolicić odpowiedź dla błędnego e-maila i hasła.
- Frontend: dodać komunikat błędu w panelu logowania.
- Frontend: zablokować wielokrotne kliknięcie przy trwającym requestcie.
- Testy jednostkowe: pokryć przypadek nieistniejącego klienta.
- Testy manualne frontend: sprawdzić błędny login, błędne hasło i poprawne logowanie.
- Bruno: dodać negatywny wariant requestu logowania lub opis scenariusza.

Suma sprintu: 24 SP

Sprint 4 - rachunki, produkty i podstawowe operacje pieniężne

Cel sprintu: udostępnić klientowi rachunki i produkty finansowe oraz operacje zmieniające saldo.

S4-01 - Otwarcie rachunku bankowego

Jako klient chcę otworzyć rachunek bankowy z nazwą, numerem, walutą i typem konta, aby mieć produkt do operacji finansowych.

Epik: E04 Story points: 8

Zadania:

- Backend: zaimplementować domenę rachunku, w tym `BankAccount`, `AccountNumber`, `BankAccountType`, `Money`.
- Backend: dodać `OpenBankAccountCommand` i handler.

- Backend: udostępnić endpoint `POST /api/accounts`.
- Baza danych: przygotować mapowanie rachunku i transakcji.
- Frontend: dodać formularz otwarcia rachunku w widoku tworzenia konta lub dashboardu.
- Testy jednostkowe: sprawdzić otwarcie rachunku i walidację wymaganych pól.
- Testy integracyjne: sprawdzić endpoint otwarcia rachunku.
- Testy manualne frontend/Bruno: wykonać request `open-account.bru` i otwarcie rachunku z UI.

S4-02 - Dodawanie produktu finansowego

Jako klient chcę dodać produkt finansowy, aby widzieć go później na dashboardzie.

Epik: E04 Story points: 8

Zadania:

- Backend: zaimplementować `AddProductCommand`, handler i generator numeru produktu.
- Backend: udostępnić endpoint `POST /api/products`.
- Baza danych: dodać mapowanie produktu, kategorii, statusu, limitu i salda początkowego.
- Frontend: przygotować modal `CreateProductModal` z wyborem kategorii, typu, waluty i salda.
- Frontend: po dodaniu produktu odświeżyć dane dashboardu.
- Testy jednostkowe: pokryć generowanie numeru produktu i poprawne dodanie produktu.
- Testy integracyjne: sprawdzić endpoint dodawania produktu.
- Testy manualne frontend/Bruno: wykonać request `add-product.bru` i dodanie produktu z dashboardu.

S4-03 - Wypłata środków z rachunku

Jako klient chcę wypłacić środki z rachunku, aby wykonać obciążenie konta.

Epik: E05 Story points: 8

Zadania:

- Backend: dodać `WithdrawMoneyCommand` i handler.
- Backend: w domenie rachunku zmniejszyć saldo i zapisać wpis transakcyjny typu debit.
- Backend: udostępnić endpoint `POST /api/accounts/{accountId}/withdraw`.
- Baza danych: utrwalić wpis transakcyjny oraz aktualne saldo.
- Frontend: przygotować akcję lub formularz wypłaty środków, jeśli ma być dostępna z UI.
- Testy jednostkowe: sprawdzić poprawną wypłatę i zapis zdarzenia domenowego.
- Testy integracyjne: sprawdzić wypłatę przez API.
- Testy manualne backend/Bruno: wykonać request `withdraw-money.bru`.

S4-04 - Wpłata przychodząca i blokada niedozwolonego salda

Jako system chcę obsłużyć wpływ przychodzący oraz zablokować wypłatę ponad saldo, aby zachować poprawność operacji finansowych.

Epik: E05 Story points: 8

Zadania:

- Backend: dodać `DepositMoneyCommand` i handler.
- Backend: udostępnić endpoint `POST /api/transfers/incoming` do symulacji przelewu przychodzącego.
- Backend: w domenie rachunku zablokować wypłatę, gdy saldo jest niewystarczające.
- Baza danych: zapisać credit/debit w transakcjach.
- Testy jednostkowe: pokryć wpłatę, kwotę ujemną i niewystarczające środki.
- Testy integracyjne: sprawdzić wpływ przychodzący i odmowę wypłaty ponad saldo.
- Testy manualne backend/Bruno: wykonać request `simulate-incoming-transfer.bru` oraz negatywny `withdraw`.

Suma sprintu: 32 SP

Sprint 5 - dashboard i model odczytowy

Cel sprintu: pokazać klientowi aktualny stan produktów, saldo łączne i ostatnie zdarzenia w panelu bankowości.

S5-01 - Endpoint dashboardu klienta

Jako klient chcę pobrać dashboard po zalogowaniu, aby szybko sprawdzić stan swoich finansów.

Epik: E06 Story points: 8

Zadania:

- Backend: dodać `GetDashboardQuery` i handler.
- Backend: udostępnić endpoint `GET /api/dashboard/{customerId}`.
- Backend: zabezpieczyć endpoint przed pobraniem danych innego klienta.
- Read model: przygotować `DashboardReadRepository`.
- Frontend: dodać hook `useDashboard` i klienta API dashboardu.
- Testy jednostkowe: pokryć handler pobierania dashboardu.
- Testy integracyjne: sprawdzić endpoint dashboardu z poprawnym i błędnym klientem.
- Testy manualne backend/Bruno: wykonać request `get-dashboard.bru`.

S5-02 - Kafelki produktów i saldo łączne w UI

Jako klient chcę widzieć kafelki produktów oraz łączne saldo, aby rozpoznać swoje rachunki i ogólny stan środków.

Epik: E06 Story points: 5

Zadania:

- Frontend: przygotować `SummarySection` z łącznym saldem i walutą.
- Frontend: przygotować `ProductsSection` z listą produktów, typem, nazwą, numerem i saldem.
- Frontend: dodać stany ładowania, pustej listy i błędu.
- Backend: upewnić się, że `DashboardResponse` zawiera pola wymagane przez UI.
- Testy manualne frontend: sprawdzić dashboard dla klienta bez produktów i z wieloma produktami.
- Testy integracyjne: sprawdzić strukturę listy produktów w odpowiedzi dashboardu.

S5-03 - Projekcje zdarzeń domenowych do read modelu

Jako system chcę aktualizować model odczytowy na podstawie zdarzeń domenowych, aby dashboard był szybki i niezależny od modelu zapisu.

Epik: E06 Story points: 13

Zadania:

- Backend: dodać zdarzenia domenowe dla utworzenia produktu, wpłaty, wypłaty i przelewu.
- Infrastructure: zapisywać zdarzenia w outboxie przy `UnitOfWork`.
- Read model: zaimplementować dispatcher projekcji.
- Read model: aktualizować `ProductTileReadModel` i `TimelineEventReadModel`.
- Baza danych: przygotować tabelę przetworzonych wiadomości, aby nie dublować projekcji.
- Testy jednostkowe: pokryć `ProjectionDispatcher` i repozytorium odczytowe.
- Testy integracyjne: sprawdzić, że operacja finansowa pojawia się na dashboardzie.
- Testy manualne backend/Bruno: wykonać sekwencję open-account, incoming-transfer, get-dashboard.

S5-04 - Oś czasu zdarzeń w dashboardzie

Jako klient chcę widzieć najnowsze zdarzenia na osi czasu, aby śledzić operacje na produktach.

Epik: E06 Story points: 5

Zadania:

- Read model: zwracać zdarzenia z datą, tytułem, kwotą, walutą i oznaczeniem wpływu/obciążenia.

- Frontend: przygotować `EventsSidebar` z listą zdarzeń.
- Frontend: odróżnić wizualnie zdarzenia dodatnie i ujemne.
- Frontend: obsłużyć pustą oś czasu.
- Testy manualne frontend: sprawdzić widoczność zdarzeń po wpłacie i wypłacie.
- Testy integracyjne: sprawdzić sortowanie i kompletność zdarzeń dashboardu.

Suma sprintu: 31 SP

Sprint 6 - przelewy własne i zewnętrzne

Cel sprintu: pozwolić klientowi wykonywać przelewy oraz odzwierciedlać je w saldach i historii.

S6-01 - Przelew własny między rachunkami klienta

Jako klient chcę wykonać przelew własny między moimi rachunkami, aby przenosić środki wewnątrz bankowości.

Epik: E05 Story points: 13

Zadania:

- Backend: zaimplementować `Transfer` typu `Own` oraz `CreateTransferCommand`.
- Backend: pobrać rachunek źródłowy i docelowy oraz sprawdzić, czy należą do klienta.
- Backend: obciążyć rachunek źródłowy i zasilić rachunek docelowy.
- Backend: oznaczyć transfer jako `completed` po poprawnym wykonaniu operacji.
- Baza danych: zapisać transfer i powiązane transakcje.
- Frontend: w panelu przelewu umożliwić wybór rachunku źródłowego i docelowego.
- Testy jednostkowe: pokryć poprawny przelew własny i brak środków.
- Testy integracyjne: sprawdzić pełny przepływ przelewu własnego przez API i dashboard.
- Testy manualne frontend/Bruno: wykonać przelew własny z UI i request `create-transfer.bru`.

S6-02 - Przelew zewnętrzny

Jako klient chcę wykonać przelew zewnętrzny na numer rachunku odbiorcy, aby zapłacić osobie lub firmie spoza moich produktów.

Epik: E05 Story points: 13

Zadania:

- Backend: zaimplementować `Transfer` typu `External`.
- Backend: walidować numer rachunku odbiorcy, nazwę odbiorcy, tytuł, kwotę i walutę.
- Backend: obciążyć rachunek źródłowy i zapisać dane odbiorcy zewnętrznego.

- Baza danych: utrwalić external target account number i recipient name.
- Frontend: dodać wariant formularza dla przelewu zewnętrznego.
- Frontend: pokazać sukces lub błąd po wysłaniu przelewu.
- Testy jednostkowe: pokryć poprawny przelew zewnętrzny i błędne dane odbiorcy.
- Testy integracyjne: sprawdzić endpoint `POST /api/transfers`.
- Testy manualne frontend/Bruno: wykonać przelew zewnętrzny w UI i Bruno.

S6-03 - Panel przelewów i szybka aktualizacja dashboardu

Jako klient chcę po przelewie od razu zobaczyć aktualne saldo i nowe zdarzenie, aby mieć pewność, że operacja została wykonana.

Epik: E05, E06 Story points: 8

Zadania:

- Frontend: przygotować `TransferPanel` z wyborem typu przelewu.
- Frontend: podłączyć `transferApi` do endpointu przelewów.
- Frontend: po sukcesie odświeżyć dane dashboardu lub wykonać lokalną aktualizację widoku.
- Frontend: wyczyścić formularz po poprawnym przelewie.
- Read model: upewnić się, że przelew generuje zdarzenie na osi czasu.
- Testy manualne frontend: sprawdzić zmianę salda bez ręcznego odświeżania strony.
- Testy integracyjne: sprawdzić, że po przelewie dashboard zwraca nowe saldo i event.

S6-04 - Walidacje i scenariusze negatywne przelewów

Jako system chcę odrzucać niepoprawne przelewy, aby nie dopuścić do utraty spójności danych finansowych.

Epik: E05 Story points: 8

Zadania:

- Backend: zablokować kwotę mniejszą lub równą zero.
- Backend: zablokować przelew z rachunku nienależącego do klienta.
- Backend: zablokować przelew przy braku środków.
- Backend: zablokować brak wymaganych pól dla przelewu zewnętrznego.
- Frontend: pokazywać błędy walidacyjne przy formularzu przelewu.
- Testy jednostkowe: pokryć najważniejsze przypadki negatywne.
- Testy integracyjne: sprawdzić statusy błędów API.
- Testy manualne backend/Bruno: przygotować negatywne warianty requestu `create-transfer.bru`.

Suma sprintu: 42 SP

Sprint 7 - stabilizacja, testy i oddanie produktu

Cel sprintu: dopracować cały przepływ klienta, usunąć niespójności i przygotować produkt do prezentacji lub odbioru.

S7-01 - Pełny przepływ klienta w UI

Jako klient chcę przejść od strony startowej przez rejestrację lub logowanie do dashboardu, aby korzystać z aplikacji bez narzędzi developerskich.

Epik: E07 Story points: 8

Zadania:

- Frontend: sprawdzić i poprawić ścieżki landing, create account, login i dashboard.
- Frontend: ujednolicić komunikaty sukcesu i błędu w formularzach.
- Frontend: zapewnić czytelne stany ładowania dla rejestracji, logowania, dashboardu i przelewów.
- Backend: sprawdzić zgodność kontraktów API z typami TypeScript.
- Testy manualne frontend: przejść scenariusz end-to-end z nowym klientem.
- Testy manualne backend/Bruno: wykonać analogiczny scenariusz przez kolekcję Bruno.

S7-02 - Responsywność i spójność wizualna

Jako klient chcę korzystać z aplikacji na komputerze i mniejszych ekranach, aby bankowość była wygodna w różnych warunkach.

Epik: E07 Story points: 5

Zadania:

- Frontend: sprawdzić layout publiczny i dashboardowy na szerokościach desktop/tablet/mobile.
- Frontend: poprawić odstępy, zawijanie tekstu i skalowanie sekcji produktów.
- Frontend: ujednolicić wygląd przycisków, pól i modali.
- Frontend: upewnić się, że błędy formularzy nie rozpychają układu.
- Testy manualne frontend: wykonać checklistę responsywności w przeglądarce.
- Dokumentacja: dopisać minimalne wymagania uruchomienia frontendu.

S7-03 - Rozszerzenie testów automatycznych backendu

Jako zespół chcemy mieć testy najważniejszych flow, aby ograniczyć ryzyko regresji przed prezentacją produktu.

Epik: E07 Story points: 13

Zadania:

- Testy jednostkowe: uzupełnić testy rejestracji, logowania, rachunków, produktów, przelewów i dashboardu.
- Testy jednostkowe: pokryć **Money**, hasher, token provider i dispatcher projekcji.
- Testy integracyjne: dodać flow rejestracja -> logowanie -> rachunek -> dashboard.
- Testy integracyjne: dodać flow rachunek -> wpływ -> wypłata -> dashboard.
- Testy integracyjne: dodać flow rachunki -> przelew -> dashboard.
- Backend: poprawić wykryte niespójności w obsłudze wyjątków i statusów HTTP.
- CI/manual: uruchomić `dotnet test` dla całego backendu.

S7-04 - Kolekcja Bruno i testy manualne API

Jako tester chcę mieć komplet requestów Bruno, aby ręcznie sprawdzić backend bez uruchamiania frontendu.

Epik: E07 Story points: 5

Zadania:

- Bruno: uporządkować foldery Customers, Accounts, Products, Transfers i Dashboard.
- Bruno: dodać przykładowe dane do rejestracji, logowania, produktu, rachunku, wpływu, wypłaty, przelewu i dashboardu.
- Bruno: opisać kolejność uruchamiania requestów w scenariuszu testowym.
- Backend: upewnić się, że endpoint `incoming` działa zgodnie z nazwą i kontraktem.
- Testy manualne backend/Bruno: wykonać pozytywną ścieżkę end-to-end.
- Testy manualne backend/Bruno: wykonać negatywne scenariusze bez tokenu, z błędną kwotą i brakiem środków.

S7-05 - Build, dokumentacja i scenariusz prezentacji

Jako zespół chcemy mieć sprawdzony build, aktualną dokumentację i scenariusz demo, aby oddać produkt w uporządkowany sposób.

Epik: E07 Story points: 5

Zadania:

- Frontend: uruchomić `npm run lint` i `npm run build`.
- Backend: uruchomić `dotnet test` i sprawdzić start API.
- Baza danych: zweryfikować, że skrypty tworzą bazy od zera.
- Dokumentacja: zaktualizować README o kolejność uruchomienia bazy, backendu, frontendu i Bruno.
- Dokumentacja: przygotować scenariusz prezentacji: rejestracja, logowanie, rachunek, wpływ, przelew, dashboard.
- Testy manualne pełne: przejść scenariusz demo od czystych danych.

Suma sprintu: 36 SP

Podsumowanie

Sprint	Zakres	SP
Sprint 1	Fundamenty architektury i środowiska	26
Sprint 2	Rejestracja klienta	21
Sprint 3	Logowanie, sesja i ochrona API	24
Sprint 4	Rachunki, produkty i podstawowe operacje pieniężne	32
Sprint 5	Dashboard i model odczytowy	31
Sprint 6	Przelewy własne i zewnętrzne	42
Sprint 7	Stabilizacja, testy i oddanie produktu	36
Razem		212

Uwagi do realizacji

- Backlog odzwierciedla funkcje widoczne w repozytorium: klientów, logowanie JWT, rachunki, produkty, przelewy, wpływ przychodzący, dashboard, outbox/projekcje, frontend React oraz Bruno.
- Największe historyjki mają 13 SP, ponieważ obejmują jednocześnie reguły domenowe, persistencję, API, frontend, projekcje i testy.
- Zadania testowe są wpisane w każdą historyjkę, aby testy jednostkowe, integracyjne oraz manualne nie były odkładane na koniec projektu.
- W kolejnych iteracjach warto rozważyć osobne epiki dla administracji, historii operacji z paginacją, przelewów cyklicznych, kart płatniczych i pożyczek, ponieważ w domenie istnieją już klasy sugerujące taki kierunek rozwoju.